

SISTEMAS OPERATIVOS

Segundo Cuatrimestre de 2026

Procesos e Hilos en C

1. Identificación de procesos padre e hijo

Uso de `fork()` para crear un nuevo proceso.

Resuelva:

- Crear un proceso hijo usando `fork()`.
- Ambos procesos (padre e hijo) deben imprimir:
 - Su propio PID.
 - El PID de su padre.
 - Un mensaje indicando si son el proceso padre o el hijo.
- Realice varias pruebas e indique si el orden en que imprimen por pantalla es siempre el mismo. Puede introducir retardos (`sleep`) para verificar.
- Extender el ejercicio anterior para que el padre espere la terminación del hijo utilizando `wait()`.
- Verificar que el mensaje final del padre se imprima después de que el hijo finaliza.

2. Creación de múltiples procesos hijos

Uso de `fork()` en un bucle.

Resuelva:

- Crear N procesos hijos (N pasado por línea de comandos).
- Cada hijo imprime su PID y el de su padre, y termina.
- El padre debe esperar la finalización de todos los hijos.
- (Extensión) Usar `waitpid()` para mayor control.

3. Ejecutar otro programa con `exec()`

Reemplazo de la imagen de un proceso.

Resuelva:

- Crear un proceso hijo con `fork()`.
- El hijo debe reemplazar su código ejecutando otro programa (por ejemplo, `ls`, `ps`, o uno creado por el alumno) usando `exec1p()` o similar.
- El padre espera a que el hijo termine y luego muestra un mensaje de finalización.

4. Introducción a `pthread_create()`

Crear y ejecutar múltiples hilos.

Resuelva:

- Crear M hilos con `pthread_create()`.
- Cada hilo imprime su número de identificación y un mensaje.
- Esperar la finalización de todos los hilos con.

5. Comparación de procesos vs. hilos

Uso de procesos e hilos para tareas concurrentes.

Resuelva:

- Implementar dos versiones de un programa que calcule la suma de dos matrices:
 1. **Versión A:** Utiliza múltiples procesos (`fork()`).
 2. **Versión B:** Utiliza múltiples hilos (`pthread_create()`).
 3. ¿Cómo resuelve la comunicación en cada caso para obtener la matriz final?

6. Ejercitación extra

1. Hijos con espera...

- Un proceso padre debe crear dos procesos hijos, uno de ellos ejecutar el comando de listar un directorio (usar el `/etc`), y el otro mostrar en pantalla un contador (de 1 a 10000). El padre deberá esperar la finalización de ambos procesos, y cuando esto ocurra lo indicará con un mensaje en pantalla.
 - Primer caso: en el proceso padre se encuentra toda la codificación (un único archivo c).
 - Segundo caso: cada proceso hijo carga una imagen ejecutable para realizar la tarea asignada (varios archivos c).

2. Grafo de tareas

El grafo representa la ejecución de varias tareas. Algunas tareas ocurren en paralelo y otras de forma secuencial, debiendo esperar en los puntos de encuentro para continuar.

- Implemente un código que resuelva el grafo mostrado. Cada proceso debe imprimir por pantalla su identificación y la de su padre.

